

6장 리액트 프로젝트 설계

이번 장은 리액트 프로젝트를 실무 수준으로 끌어올리기 위해 컴포넌트 구조 설계, 화면 흐름과 라우팅 구성까지 학습하며, 단순한 학습 예제를 넘어 실제 서비스 환경에서 유지보수성과 확장성을 고려한 프로젝트 구조를 구축하고 협업 효율성을 높이는 방법을 단계적으로 다루는 과정이다.

6.1 실무 컴포넌트 구조 설계

이번 절에서는 프로젝트의 확장성과 유지보수성을 위해 도메인 중심, 기능 중심, 화면 중심 등 다양한 폴더 구조를 비교하고, 중첩 컴포넌트 설계 전략과 협업 시 발생할 수 있는 문제를 해결할 수 있는 실무형 설계 방안을 학습하여 팀 단위로 일관성 있는 컴포넌트 구조를 구축하는 방법을 다룬다.

(1) 도메인 중심 폴더 설계 사례

도메인 중심 구조는 기능별로 분리된 폴더 대신 회원, 게시판, 상품 등 도메인 단위로 구성해 유지보수성을 높이는 방식으로, 실제 서비스 환경에서 요구되는 응집도와 재사용성을 고려해 프로젝트를 설계할 수 있는 구체적인 사례를 제시한다.

① 도메인 중심 폴더 설계

도메인 중심 폴더 구조는 기능이나 UI 레이어 대신 비즈니스 도메인(예: user, product, order) 단위로 파일과 컴포넌트를 모아 관련 코드의 응집도를 높이고, 도메인 책임을 한곳에 모아 변경에 강한 구조를 설계함으로써 대규모 프로젝트에서 유지보수성과 이해도를 크게 향상시킨다.

- **핵심 아이디어:** 관심사(Concerns)가 아닌 도메인(비즈니스 개념)별로 파일과 폴더를 그룹화.
- **장점:** 관련 컴포넌트·서비스·테스트·스타일을 한 곳에서 찾을 수 있어 기능 변경 시 영향 범위를 파악하기 쉬움.
- **폴더 구조**

```

src/
├── domains/
│   ├── user/
│   │   ├── components/
│   │   │   └── UserProfile.jsx
│   │   ├── api.js
│   │   ├── hooks.js
│   │   └── user.slice.ts
│   ├── product/
│   │   └── ...
│   └── common/
│       ├── ui/
│       └── utils/

```

- **권장 규칙:** 각 도메인 내부에 `components`, `hooks`, `api`, `styles`, `tests` 를 두어 일관성 유지.

② 도메인 경계와 응집도 설계

도메인 경계는 시스템의 비즈니스 의미에 따라 정의되어야 하며, 각 도메인은 높은 응집도와 낮은 결합도를 목표로 하여 책임이 명확한 모듈로 설계하면 기능 변경 시 영향 반경을 제한할 수 있다.

- **경계 정의 기준:** 업무 규칙, 데이터 소유권, API 경계(백엔드 엔드포인트) 기준으로 도메인을 나눔.
- **응집도 체크리스트:** 같은 도메인내 데이터와 로직이 얼마나 자주 함께 변경되는가? 변경이 자주 같이 일어나면 같은 도메인에 포함.
- **결합도 줄이기:** 도메인 간 통신은 명확한 인터페이스(api module)로만 하며, 직접 내부 구현을 참조하지 않도록 함.

③ 도메인 내부 구조 표준

각 도메인은 컴포넌트, 서비스(API), 훅, 스타일, 테스트 파일을 표준화된 서브폴더로 나누어 관리하고, 동일한 규칙을 팀 전반에 적용해 진입 장벽을 낮추는 것이 중요하다.

구체적으로 예를 들어 아래와 같이 도메인 내부 구조를 구성할 수 있습니다.

- `domains/<domain>/components/` — UI 컴포넌트 (프레젠테이션 위주)
- `domains/<domain>/hooks/` — 도메인 전용 훅 (비즈니스 로직 재사용)

- `domains/<domain>/api.js` — 백엔드 통신 함수(axios 인스턴스 사용)
- `domains/<domain>/index.js` — 외부에 노출할 퍼블릭 API(단일 진입점)

④ 도메인 중심으로 마이그레이션하는 법

기존 기능 중심 또는 화면 중심 구조에서 도메인 중심으로 전환할 때는 한 도메인씩 리팩토링하고, 인터페이스(예: `domains/<d>/index.js`)를 먼저 만들며 테스트를 보장한 상태로 점진적으로 이관하는 방식이 안전하다.

1. 우선 작은 도메인 1개 선정 (예: auth)
2. 기존 관련 파일을 도메인 폴더로 옮기고, `index.js` 로 외부 의존을 조절
3. 기존 경로를 유지하는 적응 레이어(간단한 re-export)로 점진 이관
4. 테스트/빌드 확인 후 다음 도메인 진행

(2) 기능 중심 vs 화면 중심 구조 비교

기능 중심 구조는 재사용 가능한 유틸과 컴포넌트 관리에 유리하고, 화면 중심 구조는 UI 단위 작업에 직관적이라는 장점이 있어 두 방식의 차이와 장단점을 비교하며 프로젝트 상황에 맞는 최적의 구조를 선택할 수 있도록 학습한다.

① 개념 비교

기능 중심 구조는 기능(예: auth, cart, billing)별로 코드를 모아 재사용과 테스트를 용이하게 하고, 화면 중심 구조는 페이지/뷰 단위로 코드를 모아 UI 작업에 직관적이어서 초기 개발 속도가 빠른 장점이 있으므로 프로젝트 성격에 따라 적절히 선택한다.

- **기능 중심(Feature-based):** 재사용성, 테스트 용이, 비즈니스 로직 응집. 대형 애플리케이션에 적합.
- **화면 중심(Page/View-based):** 빠른 프로토타이핑, UI 중심 작업에 유리, 소규모 앱에 적합.

② 장단점 비교

기능 중심은 확장성과 모듈성에서 우수하지만 초기 구조 설계 비용이 들고, 화면 중심은 빠르게 화면을 만들 수 있으나 코드 중복과 유지보수 부담이 증가할 수 있다.

비교 포인트

- 변경 빈도: UI 잦은 변경 → 화면 중심 선호
- 팀 규모: 여러 팀/대형 프로젝트 → 기능 중심 권장
- 테스트: 비즈니스 로직 집중 테스트 용이 → 기능 중심

③ 선택 기준과 혼용 전략

프로젝트 규모, 팀 구조, 배포 주기, 기능의 재사용성 등을 고려해 구조를 선택하고, 필요한 경우 도메인/기능/화면 관점을 혼합한 하이브리드 구조를 적용해 장점을 취합한다.

아래 내용은 혼용에 관한 예입니다.

- `src/features/`에는 기능 단위, `src/pages/`에는 라우터 기반 화면을 둬
- 공통 컴포넌트는 `src/components/ui/`에 배치

④ 실제 사례 매핑

예를 들어 전자상거래 앱은 `product`, `cart`, `checkout` 처럼 기능(도메인) 중심으로 설계하고, 마케팅이나 랜딩 페이지처럼 UI 중심인 영역은 화면 중심으로 관리하여 효율성을 높일 수 있다.

- 결제/주문 로직은 기능(도메인)으로, 콘텐츠 페이지는 화면 중심으로 관리
- 공통 규칙 문서화(README)로 팀 합의

(3) 중첩 컴포넌트 설계 전략

복잡한 UI를 효율적으로 관리하기 위해 상위 컴포넌트 안에 작은 단위의 컴포넌트를 중첩 설계하는 전략을 다루며, 각 역할을 분리하여 코드 가독성과 재사용성을 높이는 동시에 유지보수를 간소화하는 방법을 실습 중심으로 학습한다.

① 분해 단위(Granularity) 설계

컴포넌트를 어느 정도로 쪼갤지(분해 단위)를 결정하는 것은 핵심 설계인데, 한 컴포넌트가 하나의 책임을 갖도록 소규모 단위로 나누되 너무 미세하게 쪼개어 관리 비용을 키우지 않도록 균형 있게 설계해야 한다.

- **단일 책임 원칙:** 하나의 컴포넌트는 하나의 역할(렌더링, 입력 관리 등)만 담당.

- **크기 가이드:** 복잡도 기준(렌더 트리 깊이, props 수)으로 분해 판단.
- **예:** `UserCard` → `Avatar`, `UserMeta`, `ActionButtons` 로 분리.

② Container vs Presentational 패턴

Container(상태/로직 담당)와 Presentational(순수 UI) 컴포넌트로 구분하면 책임이 명확해지고 테스트와 재사용성이 좋아지며, 특히 중첩 구조에서 상위는 데이터 제공자, 하위는 UI 표시자로 역할을 분리하는 것이 유리하다.

```
// Container
function UserCardContainer({ userId }) {
  const user = useUser(userId);
  return <UserCard user={user} onFollow={...} />;
}
// Presentational
function UserCard({ user, onFollow }) { return <div>...</div>; }
```

③ Props 인터페이스 설계 & 문서화

하위 컴포넌트에 전달하는 props는 최소화하고 명확한 타입(PropTypes/TypeScript)과 문서(예: JSDoc, Storybook argTypes)를 통해 계약(contract)을 정의해 컴포넌트의 사용성을 높인다.

- props는 필요한 데이터와 콜백만 전달
- 복잡한 데이터는 객체 하나로 묶어 전달 (`config` object)
- TypeScript에서 `Props` 타입 선언 권장:

```
type UserCardProps = { user: User; onFollow: (id:number)⇒void; }
```

④ 구성(Composition) 패턴: children, render props, hooks

중첩 UI를 설계할 때는 상속이 아닌 조합(composition)을 사용해 `children`, render props, 또는 커스텀 훅으로 역할을 위임하면 유연성과 재사용성이 높아진다.

아래의 내용은 구성 패턴의 예입니다.

- `children`

```
<Modal>
  <ModalHeader/> <ModalBody/> <ModalFooter/>
</Modal>
```

- render-props / function-as-child

```
<DataProvider render={data => <UI data={data}/>} />
```

- hooks: `usePagination`, `useToggle` 등으로 로직 분리

⑤ 성능 고려 및 메모이제이션

중첩 컴포넌트가 많아지면 불필요한 렌더링이 발생하므로 `React.memo`, `useMemo`, `useCallback` 을 적절히 사용하고 props 형태(primitive vs object)도 고려해 재렌더링을 최소화해야 한다.

- 콜백은 `useCallback` 으로 메모이제이션
- 큰 데이터 계산은 `useMemo` 로 캐시
- 자식은 `React.memo` 로 감싸 변경이 있을 때만 렌더

(4) 팀 단위 협업을 고려한 설계 실습

팀 협업 환경에서는 일관된 폴더 구조와 네이밍 규칙이 중요하므로, 설계 표준을 정하고 공통 컴포넌트 분리를 통해 충돌을 줄이며 협업 효율성을 높이는 방법을 실습하여 실제 프로젝트 경험을 대비한 실무 중심 학습을 진행한다.

① 네이밍 룰과 코드 컨벤션 도입 소개

팀 협업에서는 네이밍 규칙(컴포넌트 대문자, 혹은 `use` 접두사, 파일명과 `export` 일치), 폴더 구조 규칙, 커밋 메시지 규칙 등을 문서로 정리해 합의하고 자동화(lint/prettier)로 강제하면 개발 생산성과 가독성이 크게 향상된다.

권장 규칙의 예입니다.

- 컴포넌트 파일 `PascalCase.jsx`, 혹은 `useSomething.js`, 스타일 `.module.css`
- ESLint + Prettier 설정 및 Husky로 pre-commit 후 적용

② 공통 컴포넌트 라이브러리 및 Storybook

팀 내 재사용 컴포넌트는 `packages/ui` 또는 `src/components/ui` 로 모아 Storybook으로 문서화·시연하면 재사용성, 디자인 일관성, QA 효율이 증가하며, 디자이너와 협업도 원활해진다.

- Storybook으로 각 컴포넌트 스토리 작성
- 디자인 토큰(컬러/스페이싱)을 중앙에서 관리
- 배포 가능한 내부 패키지(예: monorepo)로 구성 가능

③ Git 워크플로우와 PR 규칙

브랜치 전략(예: GitFlow 또는 trunk-based), PR 템플릿, 코드 리뷰 체크리스트(성능·접근성·테스트 포함)를 정해 코드 품질을 확보하고 병합 충돌을 줄이는 프로세스를 운영한다.

- 브랜치: `feature/*`, `fix/*`, `hotfix/*`
- PR: 변경 목적, 스크린샷, 테스트 방법 명시
- 자동 CI: lint, unit test, storybook 빌드 검사

④ 문서화·온보딩·테스트 전략

프로젝트 초기화 문서(README, ARCHITECTURE.md), 컴포넌트 사용법(Storybook), 온보딩 체크리스트, 테스트(유닛·통합) 샘플을 마련해 신규 멤버가 빠르게 적응하고 코드 품질을 일정 수준 이상 유지하도록 한다.

권장 항목

- `ARCHITECTURE.md` 에 폴더 규칙·네이밍·예시 코드 포함
- `CONTRIBUTING.md` 에 PR 규칙과 테스트 작성 가이드 포함
- CI에서 coverage 기준 설정

⑤ 충돌 방지와 대형 리팩토링 가이드

대형 리팩토링은 작은 단계로 나누어 feature-flag, 브랜치 전략, 명확한 릴리즈 노트와 테스트 커버리지를 갖춘 채로 진행하며, 변경 범위가 넓을 때는 디자인 토큰·API 인터페이스 등 계약을 먼저 고정한다.

실무 체크리스트

- 작은 스텝으로 커밋(atomic change)

- 기능 플래그로 안전한 롤아웃
- 통합 테스트 및 스테이징에서 검증

6.2 화면 흐름 및 라우팅 구성

이번 절에서는 리액트의 SPA 구조를 기반으로 React Router를 설치하고 경로를 설정하는 방법, Link와 useNavigate를 활용한 화면 이동, 중첩 라우팅 및 파라미터 처리 방식을 학습하여 다양한 화면 흐름을 유연하게 제어하고 사용자 경험을 극대화하는 라우팅 구성 능력을 배운다.

(1) SPA 구조 이해

SPA는 한 번 로드된 페이지에서 필요한 데이터만 갱신하는 방식으로 빠른 전환과 매끄러운 사용자 경험을 제공하며, 라우팅을 통해 실제 다중 페이지처럼 동작하도록 구조를 이해하는 과정이다.

① SPA 개념과 사용자 경험

SPA는 초기엔 한 번 애플리케이션을 로드하고 그 후에는 서버에서 전체 페이지가 아니라 필요한 데이터만 받아와 클라이언트에서 DOM을 갱신하므로, 빠른 화면 전환과 부드러운 사용자 경험을 제공하나 초기 번들 크기, SEO, 브라우저 히스토리 처리 등 고려해야 할 기술적 이슈도 함께 존재한다.

- 장점: 빠른 페이지 전환, 상태 보존, 풍부한 클라이언트 로직.
- 단점/대응: 초기 로드 최적화(코드 분할), SEO/메타태그(서버사이드 렌더링/프리렌더링 고려), 히스토리 API 관리 필요.
- 브라우저 히스토리: `history.pushState` 기반이며 React Router가 이를 래핑.

② 라우팅과 브라우저 히스토리

클라이언트 라우팅은 브라우저의 History API(pushState/popState)를 활용해 URL을 변경하되 전체 페이지 리로드 없이 상태만 바뀌 주소 표시줄을 업데이트하며, 뒤로/앞으로 탐색은 popstate 이벤트로 처리되어 사용자가 브라우징 경험을 자연스럽게 이어갈 수 있게 해준다.

- React Router는 내부적으로 History API를 사용하여 URL을 바꾸고 `Routes`를 재매핑함.
- `navigate(-1)`은 히스토리 스택에서 뒤로 이동.
- 팁: 페이지 전환 시 `window.scrollTo(0,0)`이나 scroll-restoration 라이브러리로 스크롤 복원 구현.

③ 초기 로딩과 코드 스플리팅

SPA의 초기 번들 크기를 줄이기 위해 페이지 단위로 코드 분할(React.lazy + Suspense, 동적 import)을 적용하면 사용자가 실제로 방문하는 경로의 코드만 로드하게 되어 초기 로드 속도를 개선할 수 있다.

코드 스니펫 (React.lazy)

```
// AppRoutes.jsx
import React, { Suspense, lazy } from 'react';
import { Routes, Route } from 'react-router-dom';
const Home = lazy(() => import('./pages/Home'));
const About = lazy(() => import('./pages/About'));

export default function AppRoutes(){
  return (
    <Suspense fallback=<div>로딩...</div>>
      <Routes>
        <Route path="/" element=<Home/> />
        <Route path="/about" element=<About/> />
      </Routes>
    </Suspense>
  );
}
```

④ SEO·접근성 고려

SPA는 클라이언트가 렌더링을 담당하기 때문에 검색엔진 최적화와 SSR을 염두에 두어야 하며, 메타태그/타이틀 업데이트, 링크에 대한 올바른 `rel` · `aria` 속성 적용 등 접근성 조치를 병행해야 한다.

- SEO: Next.js 같은 SSR/SSG 프레임워크 고려 또는 prerendering.
- 접근성: `aria-*`, semantic tags, focus 관리(페이지 전환 시 focus 이동) 등.

(2) 설치 및 경로 설정 - React Router

React Router를 설치한 뒤 Route 컴포넌트를 사용하여 경로와 컴포넌트를 연결하고, 기본 화면과 서브 화면을 구성하며 SPA에서의 라우팅 기본기를 학습한다.

① 설치 및 기본 셋업

React Router는 `npm install react-router-dom` 으로 추가하고, 앱 루트(index.js)에서 `BrowserRouter` 로 래핑해 경로별로 컴포넌트를 매핑하는 기본 구조를 만드는 것이 첫 단계이다.

명령

```
npm install react-router-dom
```

index.js

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';

createRoot(document.getElementById('root')).render(
  <BrowserRouter><App/></BrowserRouter>
);
```

② Route 구성과 기본 라우트 패턴

`Routes` 와 `Route` 를 사용하여 경로와 컴포넌트를 정의하며, 기본 경로(`/`) 리디렉션이나 와일드카드(`*`)를 사용한 404 처리 등 일반 패턴을 설정해 앱의 진입점과 예외 처리를 관리한다.

App.js

```
import { Routes, Route, Navigate } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import NotFound from './pages/NotFound';

export default function App(){
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/home" replace/>} />
      <Route path="/home" element={<Home/>} />
      <Route path="/about" element={<About/>} />
      <Route path="*" element={<NotFound/>} />
    </Routes>
  );
}
```

③ 라우트 구조 설계

라우트 설계는 앱의 정보 구조에 따라 계층적으로 설계하되, 라우트 네이밍은 RESTful 경향(리소스/ID)을 따르고, 라우트 설정은 별도 파일(route config)로 분리해 가독성과 테스트를 높이는 것이 좋다.

예시 (route config)

```
// routesConfig.js
export const routes = [
  { path: '/home', label: 'Home', element: Home },
  { path: '/products', label: 'Products', element: Products },
];
```

④ SPA에서 서버 설정 주의

클라이언트 라우팅을 사용하면 서버가 모든 경로를 루트(/index.html)로 서빙하도록 설정해야 하며, Netlify/ Vercel/Express 등 배포 환경에서 리다이렉트/rewrites를 적절히 설정해야

404 이슈를 방지할 수 있다.

예시 (Netlify `_redirects`)

```
/* /index.html 200
```

(3) 활용한 페이지 이동 - Link, useNavigate

화면 전환 시 새로고침 없이 이동할 수 있도록 Link 컴포넌트와 useNavigate hooks 사용하는 방법을 배우며, 이벤트 기반으로 경로를 제어하는 실무 활용법을 익힌다.

① Link 컴포넌트 기본 사용

Link는 앵커 태그(`<a>`) 대신 사용하는 SPA 전용 네비게이션 컴포넌트로, 페이지를 새로고침하지 않고 URL만 변경하여 해당 라우트를 렌더링하므로 사용자 경험이 매끄럽다.

예시

```
import { Link } from 'react-router-dom';
export default function Nav(){
  return (
    <nav>
      <Link to="/home">홈</Link>
      <Link to="/about">소개</Link>
    </nav>
  );
}
```

② NavLink 및 활성화 스타일링

NavLink는 활성화 여부를 알려주는 클래스나 스타일 콜백을 제공하여 현재 경로에 맞는 메뉴 하이라이트를 자동으로 적용할 수 있어 네비게이션 UI를 쉽게 구현할 수 있다.

예시

```
import { NavLink } from 'react-router-dom';
<NavLink to="/home" className={({isActive}) => isActive ? 'active' : ''}>홈
```

```
</NavLink>
```

③ `useNavigate`로 프로그래밍적 이동

`useNavigate` 혹은 이벤트나 로직(로그인 처리, 폼 제출 등)에 따라 프로그램적으로 경로 이동을 수행할 수 있게 하며, `navigate('/path', { replace: true, state: { from: 'login' } })` 같은 옵션을 제공한다.

예시

```
import { useNavigate } from 'react-router-dom';
const navigate = useNavigate();
const onLoginSuccess = () => navigate('/home', { replace: true });
```

④ `useLocation`으로 상태/이전 경로 읽기

`useLocation` 을 사용하면 `navigate` 로 전달한 `state` 나 현재 URL 정보를 읽을 수 있어 로그인 리다이렉트 복원, 모달 컨텍스트 전달 등 유연한 화면 흐름 제어가 가능하다.

예시

```
import { useLocation } from 'react-router-dom';
const { state } = useLocation();
console.log(state?.from); // 이전 경로 정보 활용
```

(4) 중첩 라우팅과 파라미터 처리

다중 단계의 화면을 구성하기 위해 부모-자식 경로를 설정하고, URL 파라미터를 통해 동적 데이터(예: 게시글 ID)를 처리하는 방법을 실습하며 실전 라우팅 구성을 완성한다.

① `Outlet`으로 중첩 라우트 구성

중첩 라우팅은 부모 라우트가 공통 레이아웃(헤더, 탭 등)을 제공하고 `Outlet` 이 자식 라우트를 렌더링하게 하여 코드 중복을 줄이고 화면 계층을 자연스럽게 표현할 수 있게 해준다.

```
// DashboardLayout.jsx
import { Outlet } from 'react-router-dom';
export default function DashboardLayout(){
  return (<div><header>대시보드</header><Outlet/></div>);
}

// App.js
<Route path="/dashboard" element={<DashboardLayout/>}>
  <Route index element={<Overview/>}/>
  <Route path="reports" element={<Reports/>}/>
</Route>
```

② URL 파라미터(useParams) 사용

동적 라우트(예: `/products/:id`)에서 `useParams` 로 경로 파라미터를 읽어 해당 리소스를 로드하거나 화면을 구성할 수 있어 리스트→상세 구조를 간단하게 구현할 수 있다.

```
import { useParams } from 'react-router-dom';
export default function ProductDetail(){
  const { id } = useParams();
  // fetch(`/api/products/${id}`) ...
  return <div>Product ID: {id}</div>;
}
```

③ 쿼리 파라미터 처리 (useSearchParams)

검색/필터 UI에서 쿼리스트링(`?q=react&page=2`)을 다루려면 `useSearchParams` 로 읽고 쓸 수 있으며, URL에 상태를 보관해 북마크/공유성을 높인다.

```
import { useSearchParams } from 'react-router-dom';
const [searchParams, setSearchParams] = useSearchParams();
const q = searchParams.get('q') || '';
// 변경: setSearchParams({ q: 'react', page: '2' })
```

④ 라우트 보호·리다이렉트와 복원 경로

인증이 필요한 페이지는 `ProtectedRoute`로 감싸서 인증 체크 후 리다이렉트하고, 로그인 후 원래 가려던 경로로 복원시키려면 `state.from` 을 저장·이용하면 자연스런 흐름을 만들 수 있다.

ProtectedRoute 예시

```
import { Navigate, useLocation } from 'react-router-dom';
function Protected({ isAuth, children }) {
  const location = useLocation();
  return isAuth ? children : <Navigate to="/login" state={{ from: location }} replace />;
}
```

6.3 리액트 프로젝트 설계 실습

이번 리액트 프로젝트 설계 실습은 도메인 중심의 폴더 구조와 라우팅·중첩 컴포넌트 설계를 실제 코드로 구현하고, 공통 컴포넌트화·상태 위치 재배치·리팩토링 기법과 코드 품질 도구를 적용해 작은 전자상거래 스타일의 실무 샘플을 완성하면서 팀 단위 운영에 적합한 설계 원칙과 실천 방법을 체계적으로 학습하도록 구성한다.

(1) 반복 구조 제거 및 공통 컴포넌트화

반복되는 UI·로직을 식별하여 공통 컴포넌트와 유틸로 추출함으로써 코드 중복을 제거하고 재사용성·테스트 용이성을 확보하며, 디자인 토큰과 소규모 UI 패턴 라이브러리를 통해 팀 단위로 일관된 UI를 유지하는 방법을 실습한다.

① 컴포넌트 추출 기준 소개: UI 혹은 로직의 재사용 가능성·변경 빈도·테스트 단위를 기준으로 컴포넌트를 추출한다.

- ② 공통 UI 라이브러리 구성: 버튼·입력·카드 같은 UI는 `src/components/ui/`에 모아 Storybook(선택)으로 문서화한다.
- ③ 유틸 함수화와 hooks 분리: 반복 네트워크/포맷 로직은 `src/utils/`와 `src/hooks/`로 분리해 재사용성을 높인다.

(2) 네이밍 룰 및 폴더 정리

일관된 네이밍 규칙(컴포넌트 PascalCase, 혹은 use 접두사, 파일명과 export 일치)과 도메인 중심 폴더 표준을 도입해 신규 개발자의 진입 비용을 낮추고 PR 리뷰 시 혼란을 줄이는 구조 정리 원칙을 적용한다.

- ① 파일/컴포넌트 네이밍 규칙: 파일명과 기본 export 이름 일치, 컴포넌트는 `PascalCase.jsx`.
- ② 폴더 정리 표준: `domains/`, `components/`, `pages/`, `services/`, `contexts/`로 역할 구분.
- ③ 코드 컨벤션 자동화: ESLint/Prettier 설정으로 스타일을 강제해 일관성 확보.

(3) 상태 위치 이동 및 분리 전략

상태(로컬 vs 전역)를 책임과 재사용성 관점에서 재배치해 불필요한 props drilling을 제거하고 Context 또는 useReducer를 통해 전역 상태를 안전하게 관리하며, 상태 위치를 옮길 때의 테스트 및 리팩토링 절차를 실습한다.

- ① 로컬 vs 전역 판별 기준: 공유 필요성과 업데이트 빈도로 전역화 여부 결정.
- ② useReducer 도입 시점: 복잡한 배열/객체 조작이 많을 때 useReducer로 명확한 액션 패턴을 적용.
- ③ Context 설계 팁: 작은 책임 단위(예: CartContext, AuthContext)로 나눠 결합도 감소.

(4) 코드 품질 향상 실습

ESLint, Prettier, 타입검사(선택), 간단한 테스트 템플릿과 CI(예: GitHub Actions) 설정으로 코드 품질 파이프라인을 구성해 코드 리뷰 전 자동 검증을 수행하고, 리팩토링 후에도 동

작을 보장하는 프로세스를 구축한다.

- ① Lint/Format 자동화: pre-commit 혹(husky)과 lint-staged로 커밋 전 검사 적용.
- ② 기본 유닛/통합 테스트 템플릿: React Testing Library로 컴포넌트 스냅샷·핸들러 테스트 예시 추가.
- ③ CI 흐름: PR시 Lint·Build·Test를 실행하는 간단한 GitHub Actions 워크플로우 구성.

(5) 프로젝트 설계

아래 내용은 프로젝트 폴더(디렉터리) 구조와 각 디렉터리 및 파일의 역할, 프로젝트 애플리케이션 생성과 라이브러리 설치 명령입니다.

① 프로젝트 디렉터리 구조

```
ch07/
├─ package.json
├─ vite.config.js
├─ index.html
├─ public/
│   └─ products.json
│   └─ users.json
└─ src/
    ├─ main.jsx
    ├─ App.jsx
    ├─ styles/
    │   └─ global.css
    ├─ layouts/
    │   └─ MainLayout.jsx
    ├─ components/
    │   ├─ Header.jsx
    │   ├─ Footer.jsx
    │   └─ ProtectedRoute.jsx
    ├─ components/ui/
    │   ├─ Button.jsx
    │   └─ Input.jsx
    └─ domains/
```

```

|   └─ product/
|       └─ ProductList.jsx
|       └─ ProductCard.jsx
|       └─ ProductDetail.jsx
└─ pages/
    └─ Home.jsx
    └─ Products.jsx
    └─ ProductPage.jsx
    └─ Cart.jsx
    └─ Login.jsx
    └─ dashboard/
        └─ DashboardLayout.jsx
        └─ Overview.jsx
        └─ Settings.jsx
└─ contexts/
    └─ CartContext.jsx
    └─ AuthContext.jsx
└─ services/
    └─ api.js
└─ utils/
    └─ jwt.js

```

- `index.html` : 앱 진입 HTML
- `vite.config.js` : Vite 설정
- `public/products.json` : 상품 데이터(모의)
- `public/users.json` : 사용자 데이터(모의)
- `src/main.jsx` : 루트 렌더링 및 Provider 래핑
- `src/App.jsx` : 라우팅 정의 (중첩 포함)
- `src/styles/global.css` : 전역 스타일
- `src/layouts/MainLayout.jsx` : 공통 레이아웃
- `src/components/Header.jsx` : 네비게이션 및 인증/카트 상태 표시
- `src/components/Footer.jsx` : 푸터
- `src/components/ProtectedRoute.jsx` : 인증 보호 라우트

- `src/components/ui/*` : 공통 UI 컴포넌트
- `src/domains/product/*` : 제품 도메인 컴포넌트 (리스트/카드/상세)
- `src/pages/*` : 페이지(라우트) 컴포넌트
- `src/contexts/CartContext.jsx` : 카트 전역 상태(useReducer + localStorage)
- `src/contexts/AuthContext.jsx` : 인증 전역 상태(localStorage + fake JWT)
- `src/services/api.js` : 서버 연동 함수(모의)
- `src/utils/jwt.js` : 학습용 JWT 헬퍼

② 설치 및 실행 명령

터미널에서

```
# 1) 폴더 생성 후 초기화(또는 복사)
# ch07 디렉터리에 아래 파일을 생성하세요.

# 2) 의존성 설치
npm init -y
npm install react react-dom react-router-dom
npm install -D vite

# (혹은 제공 package.json을 사용하면)
# npm install

# 3) 개발 서버 실행 (Vite)
npx vite
# 또는 package.json scripts 가 있으면
# npm run dev
```

(아래 package.json은 예제와 호환됩니다 — 만약 직접 파일을 만들면 scripts에 "dev": "vite" 넣어주세요.)

(6) 프로젝트 설정

프로젝트에 필요한 여러 설정과 애플리케이션 설정, 통합 설정 등에 관한 내용입니다.

① package.json

```
{
  "name": "ch07",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.14.1"
  },
  "devDependencies": {
    "vite": "^5.4.8"
  }
}
```

② vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()]
});
```

③ index.html

```

<!doctype html>
<html lang="ko">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>ch07 - 리액트 프로젝트 설계 실습</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

④ public/products.json

```

[
  { "id": 1, "title": "무선 이어폰", "price": 59900, "desc": "고음질 무선 이어폰", "image": "" },
  { "id": 2, "title": "스마트 워치", "price": 129000, "desc": "심박수·알림 지원", "image": "" },
  { "id": 3, "title": "블루투스 스피커", "price": 79000, "desc": "강력한 출력", "image": "" }
]

```

⑤ public/users.json

```

[
  { "id": 1, "username": "alice", "password": "alice123", "name": "Alice" },
  { "id": 2, "username": "bob", "password": "bob123", "name": "Bob" }
]

```

⑥ src/main.jsx

```

import React from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './styles/global.css';
import { AuthProvider } from './contexts/AuthContext';
import { CartProvider } from './contexts/CartContext';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <AuthProvider>
      <CartProvider>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </CartProvider>
    </AuthProvider>
  </React.StrictMode>
);

```

앱 진입점 — BrowserRouter, AuthProvider, CartProvider로 감쌌.

⑦ src/styles/global.css

```

:root{
  --bg:#f4f6f8;
  --card:#fff;
  --primary:#0066ff;
  --muted:#6b7280;
  --radius:8px;
}
*{box-sizing:border-box}
body{margin:0;font-family:Inter,system-ui,Arial;background:var(--bg);color:#111}
.app{max-width:1000px;margin:32px auto;padding:16px}
.header{display:flex;justify-content:space-between;align-items:center;padding:12px 16px;background:var(--card);border-radius:10px;box-shadow:0

```

```

4px 12px rgba(0,0,0,.06)}
.nav a{margin-right:12px;color:var(--muted);text-decoration:none}
.card{background:var(--card);padding:16px;border-radius:8px;box-shado
w:0 6px 18px rgba(0,0,0,.05)}
.product-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(2
20px,1fr));gap:16px}
.product-card{padding:12px;border-radius:10px;border:1px solid #eee}
.btn{background:var(--primary);color:#fff;padding:8px 12px;border-radius:
8px;border:none;cursor:pointer}
.small{font-size:13px;color:var(--muted)}

```

간단한 전역 스타일

⑧ src/App.jsx

```

import React from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';
import MainLayout from './layouts/MainLayout';
import Home from './pages/Home';
import Products from './pages/Products';
import ProductPage from './pages/ProductPage';
import Cart from './pages/Cart';
import Login from './pages/Login';
import DashboardLayout from './pages/dashboard/DashboardLayout';
import Overview from './pages/dashboard/Overview';
import Settings from './pages/dashboard/Settings';
import ProtectedRoute from './components/ProtectedRoute';

export default function App(){
  return (
    <MainLayout>
      <Routes>
        <Route path="/" element={<Navigate to="/home" replace/>}/>
        <Route path="/home" element={<Home/>}/>
        <Route path="/products" element={<Products/>}/>
        <Route path="/products/:id" element={<ProductPage/>}/>
        <Route path="/cart" element={<Cart/>}/>

```

```

    <Route path="/login" element={<Login/>}/>

    {/ Protected nested dashboard */}
    <Route element={<ProtectedRoute />}>
      <Route path="/dashboard" element={<DashboardLayout/>}>
        <Route index element={<Overview/>}/>
        <Route path="settings" element={<Settings/>}/>
      </Route>
    </Route>

    <Route path="*" element={<div style={{padding:20}}>404 - 페이지를
    찾을 수 없습니다</div> } />
  </Routes>
</MainLayout>
);
}

```

라우트 정의 — 중첩 라우팅(dashboard)과 ProtectedRoute 적용.

⑨ src/layouts/MainLayout.jsx

```

import React from 'react';
import Header from '../components/Header';
import Footer from '../components/Footer';

export default function MainLayout({ children }) {
  return (
    <div className="app">
      <Header />
      <main style={{marginTop:16}}>
        {children}
      </main>
      <Footer />
    </div>
  );
}

```

공통 레이아웃(헤더/푸터)

(7) 공통 요소 작성

① src/components/Header.jsx

```
import React from 'react';
import { Link, NavLink } from 'react-router-dom';
import { useCart } from '../contexts/CartContext';
import { useAuth } from '../contexts/AuthContext';

export default function Header(){
  const { items } = useCart();
  const { isAuthenticated, user, logout } = useAuth();
  return (
    <header className="header">
      <div style={{fontWeight:800}}><Link to="/home">ch07 샘플</Link></div>
      <nav className="nav">
        <NavLink to="/home">Home</NavLink>
        <NavLink to="/products">Products</NavLink>
        <NavLink to="/cart">Cart {items.length}</NavLink>
        <NavLink to="/dashboard">Dashboard</NavLink>
      </nav>
      <div style={{display:'flex',alignItems:'center',gap:8}}>
        {isAuthenticated ? (
          <>
            <span className="small">안녕하세요, {user.name}</span>
            <button className="btn" onClick={logout}>로그아웃</button>
          </>
        ) : (
          <Link to="/login"><button className="btn">로그인</button></Link>
        )}
      </div>
    </header>
  );
}
```

```
);  
}
```

네비게이션, 카트 개수, 로그인 상태 표시.

② src/components/Footer.jsx

```
import React from 'react';  
export default function Footer(){  
  return <footer style={{marginTop:16,textAlign:'center'}} className="small">  
    © {new Date().getFullYear()} ch07 demo</footer>;  
}
```

③ src/components/ProtectedRoute.jsx

```
import React from 'react';  
import { Outlet, Navigate, useLocation } from 'react-router-dom';  
import { useAuth } from '../contexts/AuthContext';  
  
export default function ProtectedRoute(){  
  const { isAuth } = useAuth();  
  const location = useLocation();  
  return isAuth ? <Outlet /> : <Navigate to="/login" state={{ from: location }} replace />;  
}
```

인증 검사 후 자식 라우트 렌더링 혹은 리다이렉트.

④ src/components/ui/Button.jsx

```
import React from 'react';  
export default function Button({ children, ...props }){  
  return <button className="btn" {...props}>{children}</button>  
}
```

공통 버튼

⑤ src/components/ui/Input.jsx

```
import React from 'react';
export default function Input(props){
  return <input className="card-input" style={{padding:8,borderRadius:6,
border:'1px solid #ddd'}} {...props} />
}
```

간단 입력 컴포넌트.

⑥ src/pages/Home.jsx

```
import React from 'react';
import ProductList from '../domains/product/ProductList';

export default function Home(){
  return (
    <div>
      <h1>환영합니다 — ch07 샘플</h1>
      <p className="small">도메인 중심 구조, 라우팅, 전역 상태 예제</p>
      <section style={{marginTop:16}}>
        <h2>상품</h2>
        <ProductList />
      </section>
    </div>
  );
}
```

(8) 상품 도메인 컴포넌트 작성

① src/domains/product/ProductList.jsx

```
import React, { useEffect, useState } from 'react';
import ProductCard from './ProductCard';

export default function ProductList(){
  const [products, setProducts] = useState([]);
  useEffect(() => {
    fetch('/products.json').then(r=>r.json()).then(setProducts);
  }, []);
  return (
    <div className="product-grid">
      {products.map(p=> <ProductCard key={p.id} product={p} />)}
    </div>
  );
}
```

목록 불러와 ProductCard로 렌더.

② src/domains/product/ProductCard.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
import { useCart } from '../../../contexts/CartContext';

export default function ProductCard({ product }){
  const { addItem } = useCart();
  return (
    <article className="product-card card">
      <h3>{product.title}</h3>
      <p className="small">{product.desc}</p>
      <p style={{fontWeight:700}}>{product.price.toLocaleString()}원</p>
      <div style={{display:'flex',gap:8,marginTop:8}}>
        <Link to={` /products/${product.id}`}><button className="btn">상세
      </button></Link>
        <button className="btn" onClick={() =>addItem(product)}>장바구니 담
      </button>
      </div>
    </article>
  );
}
```

```
);
}
```

단일 제품 카드 — 상세 링크와 장바구니 추가.

③ src/domains/product/ProductDetail.jsx

```
import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { useCart } from '../contexts/CartContext';

export default function ProductDetail(){
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const { addItem } = useCart();

  useEffect(() => {
    fetch('/products.json').then(r=>r.json()).then(list => {
      const p = list.find(x=>String(x.id)===String(id));
      setProduct(p || null);
    });
  },[id]);

  if(!product) return <div className="card">상품을 불러오는 중...</div>;
  return (
    <div className="card">
      <h2>{product.title}</h2>
      <p>{product.desc}</p>
      <p style={{fontWeight:700}}>{product.price.toLocaleString()}원</p>
      <button className="btn" onClick={()=>addItem(product)}>장바구니 담기</button>
    </div>
  );
}
```

파라미터를 사용해 개별 상품 조회.

④ src/pages/Products.jsx

```
import React from 'react';
import ProductList from '../domains/product/ProductList';

export default function Products(){
  return (
    <div>
      <h2>상품 목록</h2>
      <ProductList />
    </div>
  );
}
```

⑤ src/pages/ProductPage.jsx

```
import React from 'react';
import ProductDetail from '../domains/product/ProductDetail';

export default function ProductPage(){
  return (
    <div>
      <ProductDetail />
    </div>
  );
}
```

⑥ src/pages/Cart.jsx

```
import React from 'react';
import { useCart } from '../contexts/CartContext';

export default function Cart(){
  const { items, removeItem, clear } = useCart();
  const total = items.reduce((s,i) => s + (i.price*(i.qty||1)),0);
  return (
```

```

<div>
  <h2>장바구니</h2>
  {items.length===0 ? <p>비어있습니다.</p> : (
    <div>
      {items.map(it => (
        <div key={it.id} className="card" style={{display:'flex',justifyConte
nt:'space-between',alignItems:'center',marginBottom:8}}>
        <div>
          <div style={{fontWeight:700}}>{it.title}</div>
          <div className="small">{(it.price).toLocaleString()}원 x {it.qty}|
1}</div>
        </div>
        <div>
          <button className="btn" onClick={() =>removeItem(it.id)}>삭제</
button>
        </div>
      </div>
    </div>
  )}}
  <div style={{marginTop:12,fontWeight:700}}>총합: {total.toLocaleStrin
g()}원</div>
  <div style={{marginTop:8}}><button className="btn" onClick={() =>
clear()}>비우기</button></div>
</div>
)}
</div>
);
}

```

CartContext를 사용해 장바구니 항목 관리.

⑦ src/contexts/CartContext.jsx

```

import React, { createContext, useContext, useEffect, useReducer } from 'r
eact';

const CartContext = createContext();

function reducer(state, action){

```

```

switch(action.type){
  case 'INIT': return action.payload;
  case 'ADD': {
    const exists = state.find(i⇒i.id===action.payload.id);
    if(exists) return state.map(i ⇒ i.id===action.payload.id ? {...i, qty:(i.qty||
1)+1} : i);
    return [...state, {...action.payload, qty:1}];
  }
  case 'REMOVE': return state.filter(i⇒i.id!==action.payload);
  case 'CLEAR': return [];
  default: return state;
}
}

export function CartProvider({ children }){
  const [items, dispatch] = useReducer(reducer, []);

  useEffect(()⇒ {
    const saved = localStorage.getItem('ch07_cart');
    if(saved) dispatch({ type:'INIT', payload: JSON.parse(saved) });
  }, []);

  useEffect(()⇒ {
    localStorage.setItem('ch07_cart', JSON.stringify(items));
  }, [items]);

  const addItem = (product) ⇒ dispatch({ type:'ADD', payload:product });
  const removeItem = (id) ⇒ dispatch({ type:'REMOVE', payload:id });
  const clear = () ⇒ dispatch({ type:'CLEAR' });

  return <CartContext.Provider value={{ items, addItem, removeItem, clear
}}>{children}</CartContext.Provider>;
}

export function useCart(){
  const ctx = useContext(CartContext);
  if(!ctx) throw new Error('useCart must be used within CartProvider');
}

```

```
return ctx;
}
```

useReducer로 카트 상태 관리 및 localStorage 동기화.

(9) 로그인 관련 컴포넌트

① src/pages/Login.jsx

```
import React from 'react';
import LoginForm from '../services/LoginForm';
import { useAuth } from '../contexts/AuthContext';
import { useLocation, useNavigate } from 'react-router-dom';

export default function Login(){
  const { login } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();
  const from = location.state?.from?.pathname || '/home';

  const onSuccess = ({ token, user }) => {
    login({ token, user });
    navigate(from, { replace: true });
  };

  return (
    <div style={{maxWidth:420}}>
      <h2>로그인</h2>
      <LoginForm onSuccess={onSuccess} />
    </div>
  );
}
```

로그인 페이지, 성공시 원래 경로로 리다이렉트.

② src/services/LoginForm.jsx

```

import React, { useState } from 'react';
import { mockLogin } from './api';

export default function LoginForm({ onSuccess }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const submit = async (e) => {
    e.preventDefault();
    setError(null); setLoading(true);
    try {
      const res = await mockLogin({ username, password });
      onSuccess(res);
    } catch (err) {
      setError(err.message || '로그인 실패');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={submit}>
      <div style={{marginBottom:8}}>
        <input placeholder="아이디" value={username} onChange={(e) => setUsername(e.target.value)} />
      </div>
      <div style={{marginBottom:8}}>
        <input type="password" placeholder="비밀번호" value={password} onChange={(e) => setPassword(e.target.value)} />
      </div>
      {error && <div style={{color:'crimson',marginBottom:8}}>{error}</div>}
      <button className="btn" type="submit" disabled={loading}>{loading
        ? '로그인 중...' : '로그인'}</button>
    </form>
  );
}

```

```
);  
}
```

로그인 폼 — services/api.mockLogin 사용.

③ src/contexts/AuthContext.jsx

```
import React, { createContext, useContext, useEffect, useMemo, useState }  
from 'react';  
import { parseFakeJWT, isExpired } from '../utils/jwt';  
  
const AuthContext = createContext();  
const KEY = 'ch07_token';  
  
export function AuthProvider({ children }) {  
  const [token, setToken] = useState(() => localStorage.getItem(KEY));  
  const [user, setUser] = useState(() => {  
    const t = localStorage.getItem(KEY);  
    if(!t) return null;  
    if(isExpired(t)) { localStorage.removeItem(KEY); return null; }  
    return parseFakeJWT(t);  
  });  
  
  useEffect(() => {  
    if(!token) { setUser(null); localStorage.removeItem(KEY); return; }  
    if(isExpired(token)) { setToken(null); setUser(null); localStorage.removeItem(KEY); return; }  
    setUser(parseFakeJWT(token));  
  }, [token]);  
  
  const login = ({ token, user }) => {  
    localStorage.setItem(KEY, token);  
    setToken(token);  
    setUser(user);  
  };  
  
  const logout = () => { localStorage.removeItem(KEY); setToken(null); setUser(null); };  
}
```

```

const value = useMemo(() => ({ token, user, isAuth: !!token && !!user, login,
logout }), [token, user]);

return <AuthContext.Provider value={value}>{children}</AuthContext.Pro
vider>;
}

export function useAuth(){
const ctx = useContext(AuthContext);
if(!ctx) throw new Error('useAuth must be used within AuthProvider');
return ctx;
}

```

인증 상태 저장/복원. (학습용 토큰 파싱 사용)

④ src/services/api.js

```

import { createFakeJWT } from '../utils/jwt';

// Products are loaded from /products.json (public)
export async function fetchProducts(){
const res = await fetch('/products.json');
if(!res.ok) throw new Error('상품 로드 실패');
return res.json();
}

// Mock login: public/users.json
export async function mockLogin({ username, password }){
const res = await fetch('/users.json');
if(!res.ok) throw new Error('사용자 로드 실패');
const users = await res.json();
const u = users.find(x => x.username===username && x.password===pas
sword);
await new Promise(r=>setTimeout(r, 400));
if(!u) { const e = new Error('아이디 또는 비밀번호가 일치하지 않습니다'); throw
e; }
const token = createFakeJWT({ sub: u.id, username: u.username, name:
u.name }, 60*60*24);

```

```
return { token, user: { id: u.id, username: u.username, name: u.name } };
}
```

외부 데이터 fetch 및 mockLogin 함수.

⑤ src/utils/jwt.js

```
// 간단한 fake JWT 인코딩/디코딩(학습용)
export function createFakeJWT(payload = {}, expiresIn = 60*60){
  const header = { alg: 'HS256', typ: 'JWT' };
  const now = Math.floor(Date.now() / 1000);
  const p = { ...payload, iat: now, exp: now + expiresIn };
  const b = (o) => btoa(unescape(encodeURIComponent(JSON.stringify(o))));
  return `${b(header)}.${b(p)}.signature`;
}

export function parseFakeJWT(token){
  try{
    const [, payload] = token.split('.');
    return JSON.parse(decodeURIComponent(escape(atob(payload))));
  }catch(e){ return null; }
}

export function isExpired(token){
  const p = parseFakeJWT(token);
  if(!p || !p.exp) return true;
  return Math.floor(Date.now()/1000) >= p.exp;
}
```

학습용 토큰 생성/파싱.

(11) 대시보드 관련 요소 작성

① src/pages/dashboard/DashboardLayout.jsx

```
import React from 'react';
import { Outlet, Link } from 'react-router-dom';

export default function DashboardLayout(){
  return (
    <div>
      <h2>대시보드</h2>
      <nav style={{marginBottom:12}}>
        <Link to="/dashboard">Overview</Link> | <Link to="/dashboard/settings">Settings</Link>
      </nav>
      <div className="card">
        <Outlet />
      </div>
    </div>
  );
}
```

② src/pages/dashboard/Overview.jsx

```
import React from 'react';
export default function Overview(){
  return <div><h3>개요</h3><p>대시보드 개요 정보</p></div>;
}
```

③ src/pages/dashboard/Settings.jsx

```
import React from 'react';
export default function Settings(){
  return <div><h3>설정</h3><p>대시보드 설정</p></div>;
}
```

